Lab Task 1: Stock Price Monitor using AVL Tree

You are building a **real-time stock price monitoring system** for a trading platform.
The platform continuously receives updates about stocks — new ones being added, existing ones changing price, and occasionally being removed.

The system must support:

- **Fast search and updates** by stock ID.
- **Automatic rebalancing** to ensure performance remains consistent even under thousands of updates.
- **Analytical queries** such as getting all stocks within a price range or identifying the highest and lowest priced stocks.

Since stock data arrives dynamically and unpredictably, the system uses an **AVL Tree** keyed by `stockID` to maintain **O(log n)** efficiency for insertions, deletions, and lookups.

Data Model

Each node in the AVL tree represents one stock record.

```
struct Stock {
    int stockID;          // Unique key for stock (e.g., 101)
    char name[50];        // Stock name
    float price;          // Current stock price
};

struct Node {
    Stock data;
    Node* left;
    Node* right;
    int height;
};
```

Functional Requirements

1. **Insertion — `addStock()`**

- Inserts a new stock into the AVL tree using `stockID` as the key.
- Duplicate IDs are **not allowed** — reject duplicates and return `false`.
- Automatically rebalances the tree after insertion.

2. **Update — `updatePrice()`**
   - Updates the `price` of an existing stock.
   - Must preserve the AVL property.
   - If stock not found, return `false`.
3. **Deletion — `removeStock()`**
   - Removes a stock node by `stockID`.
   - Properly rebalances the tree after deletion.
4. **Find — `findStock()`**
   - Returns a pointer to the stock record matching the ID.
   - Returns `nullptr` if not found
5. **Range Query — `getStocksInPriceRange()`**
   - Returns all stocks whose `price` lies in `[minPrice, maxPrice]`.
   - Output via dynamically allocated array (created with `new`), and sets `outCount` to the number of results.
6. **Highest/Lowest Queries**
   - `getHighestPricedStock()` returns the stock with the maximum price.
   - `getLowestPricedStock()` returns the stock with the minimum price.
   - If the tree is empty, return a dummy Stock with `stockID = -1`.
7. **Diagnostics**
   - `isBalanced()` : verifies AVL property (height difference ≤ 1 for all nodes).

- `getHeight()` : returns the tree's height.
- `getTotalStocks()` : returns the total number of stocks currently stored.

8. **Display**
   - `displayInorder()` : prints stock IDs in ascending order for debugging.

Test Cases

```
#include "gtest/gtest.h"
#include "StockAVL.h"

// 1. Insert and Find Stock
TEST(StockAVLTest, AddAndFindStock) {
    StockAVL avl;
    EXPECT_TRUE(avl.addStock(101, "Tesla", 750.5));
    EXPECT_TRUE(avl.addStock(102, "Apple", 180.2));

    Stock* s = avl.findStock(101);
    ASSERT_TRUE(s != nullptr);
    EXPECT_STREQ(s->name, "Tesla");
    EXPECT_FLOAT_EQ(s->price, 750.5);
}

// 2. Reject Duplicate Stock IDs
TEST(StockAVLTest, RejectDuplicateStockIDs) {
    StockAVL avl;
    EXPECT_TRUE(avl.addStock(100, "Microsoft", 320.1));
    EXPECT_FALSE(avl.addStock(100, "Meta", 240.0));
    EXPECT_EQ(avl.getTotalStocks(), 1);
}

// 3. Update Existing Stock Price
TEST(StockAVLTest, UpdateStockPrice) {
    StockAVL avl;
    avl.addStock(1, "NVIDIA", 100.0);
    EXPECT_TRUE(avl.updatePrice(1, 120.0));

    Stock* s = avl.findStock(1);
```

```
    ASSERT_TRUE(s != nullptr);
    EXPECT_FLOAT_EQ(s->price, 120.0);
}

// 4. Update Non-Existing Stock Fails
TEST(StockAVLTest, UpdateNonExistingStock) {
    StockAVL avl;
    avl.addStock(5, "Intel", 35.0);
    EXPECT_FALSE(avl.updatePrice(10, 90.0)); // no stock with ID 10
}

// 5. Auto-Balancing After Multiple Inserts (LR Rotation)
TEST(StockAVLTest, AutoBalancingAfterInsert) {
    StockAVL avl;
    avl.addStock(30, "A", 10.0);
    avl.addStock(10, "B", 20.0);
    avl.addStock(20, "C", 30.0); // triggers LR rotation
    EXPECT_TRUE(avl.isBalanced());
}

// 6. Delete Stock and Maintain Balance
TEST(StockAVLTest, DeleteAndRebalance) {
    StockAVL avl;
    avl.addStock(50, "A", 10.0);
    avl.addStock(40, "B", 15.0);
    avl.addStock(60, "C", 20.0);

    EXPECT_TRUE(avl.removeStock(40));
    EXPECT_TRUE(avl.isBalanced());
    EXPECT_EQ(avl.getTotalStocks(), 2);
}

// 7. Remove Non-Existing Stock
TEST(StockAVLTest, RemoveNonExistingStock) {
    StockAVL avl;
    avl.addStock(10, "A", 100.0);
    EXPECT_FALSE(avl.removeStock(99));
    EXPECT_EQ(avl.getTotalStocks(), 1);
}
```

```
// 8. Find Highest and Lowest Priced Stocks
TEST(StockAVLTest, HighestLowestStock) {
    StockAVL avl;
    avl.addStock(1, "X", 50.0);
    avl.addStock(2, "Y", 100.0);
    avl.addStock(3, "Z", 75.0);

    Stock high = avl.getHighestPricedStock();
    Stock low  = avl.getLowestPricedStock();

    EXPECT_STREQ(high.name, "Y");
    EXPECT_STREQ(low.name, "X");
}

// 9. Get Stocks in Price Range
TEST(StockAVLTest, StocksInPriceRange) {
    StockAVL avl;
    avl.addStock(1, "A", 10.0);
    avl.addStock(2, "B", 50.0);
    avl.addStock(3, "C", 70.0);
    avl.addStock(4, "D", 100.0);

    int count = 0;
    Stock* results = avl.getStocksInPriceRange(40.0, 80.0, count);
    EXPECT_EQ(count, 2); // Stocks B and C
    EXPECT_TRUE(results[0].price >= 40.0 && results[0].price <= 80.0);
    EXPECT_TRUE(results[1].price >= 40.0 && results[1].price <= 80.0);
    delete[] results;
}

// 10. Tree Height After Multiple Insertions
TEST(StockAVLTest, HeightAfterInsertions) {
    StockAVL avl;
    for (int i = 1; i <= 15; ++i)
        avl.addStock(i, "S", (float)(i * 10));
    EXPECT_TRUE(avl.isBalanced());
    EXPECT_LT(avl.getHeight(), 8);
}

// 11. Deletion Sequence Maintains Balance
```

```
TEST(StockAVLTest, SequentialDeletionsBalanced) {
    StockAVL avl;
    for (int i = 1; i <= 10; ++i)
        avl.addStock(i, "P", (float)(i * 5));
    for (int i = 2; i <= 10; i += 2)
        avl.removeStock(i);
    EXPECT_TRUE(avl.isBalanced());
}

// 12. Delete All Stocks
TEST(StockAVLTest, DeleteAllStocks) {
    StockAVL avl;
    avl.addStock(10, "A", 20.0);
    avl.addStock(20, "B", 30.0);
    avl.addStock(30, "C", 40.0);

    avl.removeStock(10);
    avl.removeStock(20);
    avl.removeStock(30);

    EXPECT_EQ(avl.getTotalStocks(), 0);
    EXPECT_TRUE(avl.isBalanced());
}

// 13. Empty Tree Highest/Lowest
TEST(StockAVLTest, EmptyTreeEdgeCase) {
    StockAVL avl;
    Stock high = avl.getHighestPricedStock();
    Stock low = avl.getLowestPricedStock();
    EXPECT_EQ(high.stockID, -1);
    EXPECT_EQ(low.stockID, -1);
}
```

Lab Task 2 Self-Balancing Leaderboard System (AVL Tree)

You are building a **real-time leaderboard system** for a gaming platform.
Players earn and lose points continuously as they play, and the system must instantly reflect changes while keeping the leaderboard sorted by **score** in descending order.

Since players' scores change dynamically and thousands of updates occur per minute, using an ordinary list or array would become inefficient.
To handle this, the leaderboard uses a **self-balancing AVL Tree**, which automatically maintains order and guarantees **O(log n)** time complexity for insertions, deletions, and updates.

Data Model

Each player record is represented by a struct:

```
struct Player {
    int playerID;        // Unique identifier
    char name[50];       // Player name
    int score;           // Player's current score
};
```

Each AVL node stores one `Player` record, plus height and balancing metadata:

```
struct Node {
    Player data;
    Node* left;
    Node* right;
    int height;
};
```

Note: The AVL Tree is ordered by score (descending).
If two players have the same score, use `playerID` to break ties (lower ID first).

Functional Requirements

*1. addPlayer(int id, const char name, int score)**

- Inserts a new player based on their `score` (higher score = higher rank).
- The `playerID` must be unique — reject duplicates and return `false`.

- The tree must automatically rebalance after insertion.
- Ordering Rule:
    - Higher `score` goes to **left** subtree.
    - Lower `score` goes to **right** subtree.
    - Equal score → compare `playerID`.

2. removePlayer(int id)

- Removes the player matching the given `playerID`.
- After removal, the tree must remain balanced.
- Returns `false` if the player is not found.

3. updateScore(int id, int newScore)

- Updates an existing player's score.
- Implemented as "remove old record → reinsert new record".
- Ensures tree remains AVL-balanced.
- Returns `false` if player not found.

4. findPlayer(int id)

- Searches the tree by `playerID` and returns a pointer to the `Player` struct.
- Returns `nullptr` if not found.

5. getTopPlayer()

- Returns the player with the **highest score**

6. getLowestPlayer()

- Returns the player with the **lowest score** (rightmost node).

7. getTopKPlayers(int k, int& outCount)

- Returns the **top K players** as a dynamically allocated array.
- Uses an in-order traversal (descending order).
- Sets `outCount = actual number of players returned`.
- Caller must `delete[]` the array.

8. getRank(int id)

- Returns the **rank** (1 = highest score).
- The rank is computed based on number of players with strictly higher scores.
- Implemented by traversing and counting during search.
- Returns `-1` if player not found.

9. Diagnostics

- `isBalanced()` → verifies AVL property.

- `getHeight()` → returns tree height.
- `getTotalPlayers()` → total number of nodes.

10. Display

- `displayDescending()` → prints all players (name, score) in descending score order.

Test Cases

```
#include "gtest/gtest.h"
#include "LeaderboardAVL.h"

// 1. Add and Find Player
TEST(LeaderboardAVLTest, AddAndFindPlayer) {
    LeaderboardAVL avl;
    EXPECT_TRUE(avl.addPlayer(101, "Alice", 250));
    EXPECT_TRUE(avl.addPlayer(102, "Bob", 400));

    Player* p = avl.findPlayer(102);
    ASSERT_TRUE(p != nullptr);
    EXPECT_STREQ(p->name, "Bob");
    EXPECT_EQ(p->score, 400);
}

// 2. Reject Duplicate Player IDs
TEST(LeaderboardAVLTest, RejectDuplicateIDs) {
    LeaderboardAVL avl;
    EXPECT_TRUE(avl.addPlayer(1, "Player1", 100));
    EXPECT_FALSE(avl.addPlayer(1, "Player2", 200));
    EXPECT_EQ(avl.getTotalPlayers(), 1);
}

// 3. Auto-Balancing After Insertions
TEST(LeaderboardAVLTest, BalancingAfterInserts) {
```

```
    LeaderboardAVL avl;
    avl.addPlayer(1, "A", 300);
    avl.addPlayer(2, "B", 500);
    avl.addPlayer(3, "C", 400); // triggers rotation
    EXPECT_TRUE(avl.isBalanced());
}

// 4. Update Score Changes Rank
TEST(LeaderboardAVLTest, UpdateScoreAffectsRank) {
    LeaderboardAVL avl;
    avl.addPlayer(1, "A", 100);
    avl.addPlayer(2, "B", 300);
    avl.addPlayer(3, "C", 200);

    int rankBefore = avl.getRank(1); // A = lowest
    avl.updateScore(1, 350);         // A moves up
    int rankAfter = avl.getRank(1);

    EXPECT_GT(rankBefore, rankAfter); // rank improves
    EXPECT_TRUE(avl.isBalanced());
}

// 5. Remove Player and Maintain Balance
TEST(LeaderboardAVLTest, DeleteAndRebalance) {
    LeaderboardAVL avl;
    avl.addPlayer(1, "X", 100);
    avl.addPlayer(2, "Y", 200);
    avl.addPlayer(3, "Z", 300);

    EXPECT_TRUE(avl.removePlayer(2));
    EXPECT_TRUE(avl.isBalanced());
    EXPECT_EQ(avl.getTotalPlayers(), 2);
}

// 6. Highest and Lowest Player
TEST(LeaderboardAVLTest, TopAndLowestPlayers) {
    LeaderboardAVL avl;
    avl.addPlayer(1, "A", 100);
    avl.addPlayer(2, "B", 500);
    avl.addPlayer(3, "C", 300);
```

```
    Player top = avl.getTopPlayer();
    Player low = avl.getLowestPlayer();

    EXPECT_STREQ(top.name, "B");
    EXPECT_STREQ(low.name, "A");
}

// 7. Top K Players Retrieval
TEST(LeaderboardAVLTest, GetTopKPlayers) {
    LeaderboardAVL avl;
    avl.addPlayer(1, "A", 100);
    avl.addPlayer(2, "B", 500);
    avl.addPlayer(3, "C", 300);
    avl.addPlayer(4, "D", 400);

    int count = 0;
    Player* top = avl.getTopKPlayers(3, count);
    EXPECT_EQ(count, 3);
    EXPECT_STREQ(top[0].name, "B"); // Highest score first
    EXPECT_STREQ(top[1].name, "D");
    EXPECT_STREQ(top[2].name, "C");
    delete[] top;
}

// 8. Get Rank Functionality
TEST(LeaderboardAVLTest, RankCalculation) {
    LeaderboardAVL avl;
    avl.addPlayer(1, "A", 100);
    avl.addPlayer(2, "B", 400);
    avl.addPlayer(3, "C", 300);

    EXPECT_EQ(avl.getRank(2), 1); // B highest
    EXPECT_EQ(avl.getRank(3), 2); // C second
    EXPECT_EQ(avl.getRank(1), 3); // A lowest
}

// 9. Delete Non-Existent Player
TEST(LeaderboardAVLTest, DeleteNonExisting) {
    LeaderboardAVL avl;
```

```
    avl.addPlayer(1, "A", 100);
    EXPECT_FALSE(avl.removePlayer(999));
    EXPECT_EQ(avl.getTotalPlayers(), 1);
}

// 10. Large Sequence Stress Test
TEST(LeaderboardAVLTest, LargeInsertDeleteSequence) {
    LeaderboardAVL avl;
    const int N = 50;
    for (int i = 1; i <= N; ++i)
        avl.addPlayer(i, "P", i * 10);
    for (int i = 1; i <= N; i += 3)
        avl.removePlayer(i);
    EXPECT_TRUE(avl.isBalanced());
}
```